EXPERIMENT NO: 14 **Date:** _____

NUMPY

AIM

Write a program to create a matrix using numpy and do the following operations: Perform matrix addition, subtraction and multiplication, Find transpose of matrix, Calculates the inverse of matrix, Calculates the determinant of matrix, Transforms a matrix into 1d array (flatten)

OBJECTIVES

- To utilize NumPy library for numerical array processing.
- To perform fundamental matrix operations (addition, subtraction, dot product).
- To compute linear algebra functions (transpose, inverse, determinant, flattening).

ALGORITHM

Step 1 :- Initialize matrix computation module using NumPy library for matrix arithmetic and linear algebra operations.

Step 2 :- Import numpy module under standard alias np to access array creation and matrix functions.

Step 3 :- Create two 2D arrays matrix1 and matrix2 with 2x2 integer values for calculations.

Step 4 :- Perform element wise addition, subtraction, and dot product multiplication using NumPy built in functions.

Step 5 :- Compute matrix transpose using np.transpose function to swap rows and columns of matrix1.

Step 6 :- Compute matrix inverse using np.linalg.inv and matrix determinant using np.linalg.det on matrix1.

Step 7 :- Transform 2D array matrix1 into 1D array flattened using matrix flatten method call.

Step 8 :- Print original matrices and calculated addition, subtraction, multiplication, transpose, inverse, determinant, and flattened outputs.

Step 9 :- Conclude NumPy matrix operations program execution and free array memory allocations from environment heap.

PROGRAM

```
import numpy as np

# Create matrices
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# Perform matrix addition
addition = np.add(matrix1, matrix2)

```

```
# Perform matrix subtraction
subtraction = np.subtract(matrix1, matrix2)

# Perform matrix multiplication
multiplication = np.dot(matrix1, matrix2)

# Find transpose of matrix1
transpose = np.transpose(matrix1)

# Calculate the inverse of matrix1
inverse = np.linalg.inv(matrix1)

# Calculate the determinant of matrix1
determinant = np.linalg.det(matrix1)

# Transform matrix1 into a 1D array (flatten)
flattened = matrix1.flatten()

# Display results
print("Matrix 1:\n", matrix1)
print("\nMatrix 2:\n", matrix2)
print("\nMatrix Addition:\n", addition)
print("\nMatrix Subtraction:\n", subtraction)
print("\nMatrix Multiplication:\n", multiplication)
print("\nTranspose of Matrix 1:\n", transpose)
print("\nInverse of Matrix 1:\n", inverse)
print("\nDeterminant of Matrix 1:\n", determinant)
print("\nFlattened Matrix 1:\n", flattened)
```

SAMPLE OUTPUT

Terminal Output

```
Matrix 1:
[1 2

4
]


Matrix 2:
[5 6

8

]


Matrix Addition:
[ 6 8

12

]


Matrix Subtraction:
[-4 -4

-4
]


Matrix Multiplication:
[19 22

50









]


Transpose of Matrix 1:
[1 3

4
]


Inverse of Matrix 1:
[-2.  1.

-0.5
]


Determinant of Matrix 1:
-2.0


Flattened Matrix 1:
2 3 4
```

RESULT

The matrix operations program using NumPy library was executed and results verified.

EXPERIMENT NO: 15 **Date:** _____

PANDA MODULE

AIM

Write a python program to read a .csv file using panda module and print the first and last five lines of a file

OBJECTIVES

- To leverage Pandas library for structured data manipulation.
- To load CSV datasets into DataFrames using $read_csv$.
- To inspect head and tail slices of dataset files.

ALGORITHM

Step 1 :- Initialize data processing program using Pandas module to read CSV file records and preview data lines.

Step 2 :- Import pandas library under alias pd to utilize DataFrame loading and inspection utility functions.

Step 3 :- Read external CSV dataset file using pd.$read_csv function and store data in DataFrame df$.

Step 4 :- Call head method on DataFrame df to retrieve the first five rows of CSV dataset.

Step 5 :- Call tail method on DataFrame df to retrieve the last five rows of CSV dataset.

Step 6 :- Print first five lines and last five lines of dataset to standard terminal console display.

Step 7 :- Terminate execution of Pandas file processing script and clear DataFrame structures from system memory.

PROGRAM

```
import pandas as pd

filename = 'example.csv'
df = pd.read_csv(filename)

print("First five lines of the file:")
print(df.head())

print("\nLast five lines of the file:")
print(df.tail())
```

SAMPLE OUTPUT

Terminal Output

```
First five lines of the file:
Name Age City
0 Alice 30 New York
1 Bob 25 Los Angeles
2 Charlie 35 Chicago
3 David 40 Houston
4 Eve 28 Phoenix

Last five lines of the file:
Name Age City
5 Frank 33 Philadelphia
6 Grace 45 San Antonio
7 Hannah 27 San Diego
8 Ivan 50 Dallas
9 Judy 29 San Jose
```

RESULT

The Pandas CSV data reading and head/tail preview program was verified successfully.

EXPERIMENT NO: 16 **Date:** _____

MATPLOTLIB

AIM

Write a program to draw geometric shapes using matplotlib

OBJECTIVES

- To initialize Matplotlib figure and subplot axis objects.
- To render geometric patch primitives (Rectangle, Circle, Polygon).
- To format plot aesthetics including labels, title, grid, aspect ratio, and show display.

ALGORITHM

Step 1 :- Initialize graphic visualization program using Matplotlib module to render geometric shapes on plot canvas.

Step 2 :- Import matplotlib.pyplot module under alias plt and numpy module under alias np for drawing.

Step 3 :- Create figure and subplots axis objects using plt.subplots function to define plot drawing canvas.

Step 4 :- Create Rectangle patch specifying position, width, height, color, and alpha transparency, adding to axis.

Step 5 :- Create Circle patch specifying center position, radius, color, and transparency, adding to plot axis.

Step 6 :- Create Polygon patch with NumPy array triangle coordinates, adding red triangle patch to axis.

Step 7 :- Set x and y axis limits from zero to one and set equal aspect ratio scaling.

Step 8 :- Set grid, axis labels, title, and display rendered figure window using plt.show function.

Step 9 :- Close graphic visualization window and terminate program execution releasing plot resources back to system.

PROGRAM

```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots()

rectangle = plt.Rectangle((0.2, 0.5), 0.4, 0.3, color='blue', alpha=0.3)
ax.add_patch(rectangle)

circle = plt.Circle((0.8, 0.2), 0.1, color='green', alpha=0.5)
ax.add_patch(circle)

```

```
triangle_points = np.array([[0.6, 0.8], [0.8, 0.8], [0.7, 0.9], [0.6, 0.8]])
triangle = plt.Polygon(triangle_points, closed=True, color='red', alpha=0.4)
ax.add_patch(triangle)

ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.set_aspect('equal', adjustable='box')

ax.grid(True)
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.set_title('Geometric Shapes')

plt.show()
```

SAMPLE OUTPUT

Terminal Output

```
[Plot Window Displayed:  Rendered Blue Rectangle, Green Circle, and Red Triangle
shapes on grid axes]
```

RESULT

The Matplotlib geometric shape rendering program was successfully executed and visualized.